

# XRComm FCS Wideband Receive Platform

## Quick Start & User Guide

XRComm Inc.

June 2026

# **XRComm FCS Wideband Receive Platform**

## Quick Start & User Guide

|                       |                                      |
|-----------------------|--------------------------------------|
| <b>Platform</b>       | XRComm FCS Wideband Receive Platform |
| <b>Driver Version</b> | v1.4.3                               |
| <b>Variant</b>        | Wideband                             |
| <b>Audience</b>       | End users and system administrators  |
| <b>Classification</b> | Proprietary                          |

© 2026 XRComm Inc. – All Rights Reserved.

## Contents

|                                                 |           |
|-------------------------------------------------|-----------|
| <b>Terminology</b>                              | <b>3</b>  |
| <b>System Topology</b>                          | <b>3</b>  |
| <b>Package Contents</b>                         | <b>4</b>  |
| <b>Installation</b>                             | <b>5</b>  |
| Extract the Package . . . . .                   | 5         |
| Run the Installer . . . . .                     | 5         |
| Uninstallation . . . . .                        | 5         |
| <b>Verifying the Services</b>                   | <b>5</b>  |
| <b>Controlling the Platform over gRPC</b>       | <b>6</b>  |
| Endpoint . . . . .                              | 6         |
| Client Setup . . . . .                          | 6         |
| Control Commands . . . . .                      | 6         |
| Worked Examples . . . . .                       | 7         |
| Telemetry Fields . . . . .                      | 7         |
| Mode Rules . . . . .                            | 7         |
| <b>Recording Captures and Reading Them Back</b> | <b>8</b>  |
| Automatic Read-Back Configuration . . . . .     | 8         |
| Setting Read-Back Parameters via gRPC . . . . . | 8         |
| Recording Workflow . . . . .                    | 8         |
| Output Files . . . . .                          | 8         |
| Offline NVMe Retrieval . . . . .                | 8         |
| <b>Troubleshooting</b>                          | <b>10</b> |
| <b>Appendix A: Environment Prerequisites</b>    | <b>10</b> |
| Dependency Version Check Table . . . . .        | 10        |
| Hugepages . . . . .                             | 11        |
| Device Binding (DPDK / SPDK) . . . . .          | 11        |
| Privileges . . . . .                            | 11        |

## Terminology

The following terms are used throughout this document.

| Term                        | Meaning                                                                                                                                                                      |
|-----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>FlexServer</b>           | The physical host machine running the XRComm platform driver and gRPC control service. All platform binaries execute here.                                                   |
| <b>Client Computer</b>      | Any machine (including the FlexServer itself) from which you run the Python gRPC control client to operate the platform.                                                     |
| <b>Platform Driver</b>      | The <code>xrcomm-server</code> binary. Receives the high-speed RF stream from the NIC, manages shared memory, records to NVMe, and dispatches data to customer applications. |
| <b>Control Service</b>      | The <code>xrcomm-grpc-ctrl</code> binary. Provides a gRPC interface (TCP port 50051) for remotely controlling the Platform Driver.                                           |
| <b>Customer Application</b> | Your own software (or the provided Hello World examples) that connects to the Platform Driver via shared memory to consume IQ samples or full packets.                       |
| <b>IQ Delivery</b>          | Mode in which interleaved int16 I/Q sample batches are dispatched to connected applications via zero-copy shared memory.                                                     |
| <b>Full-Packet Delivery</b> | Mode in which raw network packets are forwarded to a single application using DPDK secondary process attachment. Requires <code>sudo</code> .                                |
| <b>NVMe Recording</b>       | Mode in which the received RF stream is written directly to the NVMe drive via SPDK for later offline extraction.                                                            |

## System Topology

The platform operates as a set of background services on the FlexServer. Remote or local clients can control the system over the network using a Python gRPC client.

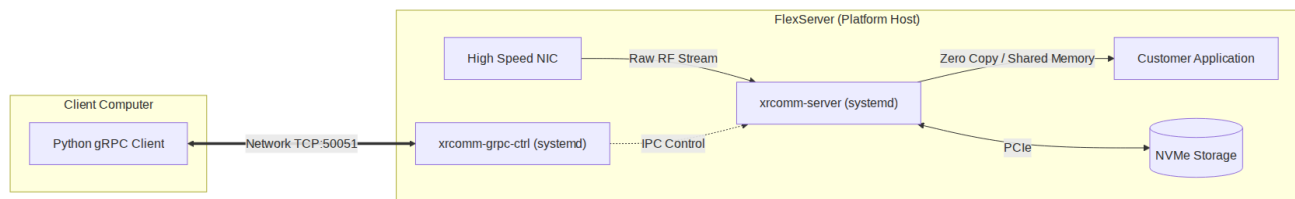


Figure 1: XRComm FCS Wideband Platform Topology

### Data flow:

1. The **High Speed NIC** receives the raw RF stream and delivers it to the **Platform Driver** via DPDK.
2. The **Platform Driver** dispatches IQ samples to connected **Customer Applications** via zero-copy shared memory.
3. Simultaneously, the Platform Driver can record the stream to the **NVMe Storage** via SPDK.
4. The **Control Service** communicates with the Platform Driver over IPC, allowing remote management.
5. The **Python gRPC Client** (on a **Client Computer** or on the FlexServer itself) connects to the Control Service over TCP port 50051.

## Package Contents

After extracting the tarball, you will find the following directory structure:

```
XRComm_FCS_platform/
  install.sh                # Automated installer
  uninstall.sh              # Clean uninstaller
  LICENSE                   # Software license agreement
  VERSION                   # Release version identifier
  CHANGELOG.md              # Release notes
  Wideband_Quickstart.pdf   # This document

XRComm_FCS_wideband_platform/
  XRComm_platform_drivers/
    bin/                    # Precompiled platform driver binary
    include/                # Public C headers for app development
    service/                # Systemd unit file templates
  XRComm_platform_gRPC/
    bin/                    # Precompiled gRPC control service binary
    client/                 # Python gRPC client scripts
    proto/                  # Protobuf service definition (.proto)
    service/                # Systemd unit file templates

XRComm_wideband_hello_world/
  hello_world.c             # IQ delivery example (no sudo)
  hello_world_fullpkt.c     # Full-packet delivery example (sudo)
  CMakeLists.txt            # Build configuration
  bin/                      # Output directory for compiled examples

XRComm_wideband_playback/
  bin/                      # Precompiled playback binary
  inputs/                   # Playback config and waveform files

XRComm_NVMe_loader/
  bin/                      # Precompiled NVMe reader binary
  config/                   # Read-back config (read_config.ini)
  scripts/                  # Interactive extraction (read_nvme.sh)
```

## Installation

### Extract the Package

Transfer the tarball to your FlexServer and extract it:

```
tar -xzf XRComm_FCS_platform.tar.gz
cd XRComm_FCS_platform
```

### Run the Installer

```
sudo ./install.sh
```

When prompted, accept the license agreement and select **Option 1** for the Wideband Platform.

The installer performs the following actions automatically:

| Step                     | What it does                                                                       |
|--------------------------|------------------------------------------------------------------------------------|
| Dependency installation  | Installs python3-grpcio, python3-protobuf, and python3-grpc-tools via apt.         |
| Systemd service creation | Creates and enables xrcomm-server.service and xrcomm-grpc-ctrl.service.            |
| Header file mapping      | Symlinks the public C headers into /usr/include/xrcomm-wideband/.                  |
| Proto compilation        | Generates the Python gRPC stubs (pipeline_control_pb2.py) in the client directory. |
| Service startup          | Starts both services immediately.                                                  |

### Uninstallation

```
sudo ./uninstall.sh
```

This stops and disables the systemd services, removes the header symlinks, and uninstalls the gRPC Python packages.

### Verifying the Services

After installation, both platform services run in the background. Verify their status:

```
systemctl status xrcomm-server.service
systemctl status xrcomm-grpc-ctrl.service
```

Both should report active (running).

To inspect live logs:

```
journalctl -u xrcomm-server.service -f      # Platform Driver logs
journalctl -u xrcomm-grpc-ctrl.service -f    # Control Service logs
```

To restart the services after a configuration change:

```
sudo systemctl restart xrcomm-server.service
sudo systemctl restart xrcomm-grpc-ctrl.service
```

## Controlling the Platform over gRPC

The remote control client is designed to run from the **Client Computer**.

### Endpoint

The Control Service listens on 0.0.0.0:50051 by default. It starts automatically as a systemd service and does not need to be launched manually.

### Client Setup

The Python gRPC client can be run from **any machine** (specifically the **Client Computer**) that has network access to the FlexServer:

- **From the FlexServer itself** (via loopback 127.0.0.1) for internal verification during installation.
- **From a remote Client Computer** for standard operational use.

Install the Python dependencies once per machine:

```
cd XRComm_FCS_wideband_platform/XRComm_platform_gRPC/client
pip3 install -r requirements.txt
```

The installer already generates the protobuf Python bindings on the FlexServer. If you are setting up the client on a separate machine, copy the `client/` directory there and generate the stubs:

```
python3 -m grpc_tools.protoc -I. --python_out=. \
  --grpc_python_out=. pipeline_control.proto
```

### Control Commands

Usage syntax:

```
python3 xrcomm_grpc_client.py --host <server-ip>:50051 <command> [value]
```

Replace <server-ip> with the IP address of your FlexServer (use 127.0.0.1 when running locally).

| Command                                              | What it does                                                                             |
|------------------------------------------------------|------------------------------------------------------------------------------------------|
| get-flags                                            | Show current delivery and recording mode flags.                                          |
| set-ip on off                                        | Turn IQ-sample delivery to applications on or off.                                       |
| set-dsp on off                                       | Turn full-packet delivery to applications on or off.                                     |
| set-logging on off                                   | Start or stop NVMe recording. Stopping triggers automatic read-back (Section 7).         |
| set-read-capture <seconds><br>[--start-offset <sec>] | Set read-back duration (seconds) and optional start offset from capture start (seconds). |
| set-read-rate <hz>                                   | Set the receiver sample rate, in Hz, used to size the read-back.                         |
| get-stats                                            | Show pipeline counters and platform telemetry.                                           |
| get-mode-stats                                       | Show received/processed/dropped counts per delivery mode and per application.            |
| list-ips                                             | List the applications currently connected to the platform.                               |
| watch-status                                         | Stream live status once per second (Ctrl-C to stop).                                     |
| shutdown                                             | Gracefully shut the platform down.                                                       |

## Worked Examples

```
C="python3 xrcomm_grpc_client.py --host 192.168.137.217:50051"
```

```
$C get-flags           # Read current modes
$C set-ip on          # Start IQ delivery
$C set-dsp on         # Start full-packet delivery
$C set-read-rate 800000000 # Set receiver rate for read-back sizing
$C set-read-capture 1.5 --start-offset 0.5 # Set readback (1.5s duration, 0.5s start offset)
$C set-logging on     # Begin recording to NVMe
$C set-logging off    # Stop recording -> auto read-back starts
$C list-ips          # Confirm your application is connected
$C get-stats         # Pipeline + platform telemetry
$C get-mode-stats    # Per-mode and per-application counters
$C watch-status      # Live 1 Hz stream (Ctrl-C to stop)
$C shutdown         # Graceful shutdown
```

## Telemetry Fields

The get-stats and watch-status commands report platform telemetry:

| Field                       | Meaning                                           |
|-----------------------------|---------------------------------------------------|
| hsp_port                    | High-speed receive port in use.                   |
| samples_received            | Total samples received from the interface.        |
| samples_logged              | Samples written to NVMe (zero unless recording).  |
| samples_processed_secondary | Samples consumed by your application(s).          |
| packets_dropped             | Receive-overflow and app-backlog drops, combined. |

## Mode Rules

- The three delivery/recording modes (IQ, full-packet, and recording) are independent and may be enabled in any combination.
- Full-packet delivery serves one application at a time. IQ delivery may be shared by multiple applications.

## Recording Captures and Reading Them Back

### Automatic Read-Back Configuration

Three values control the automatic read-back. They are set from the control client and stored in `XRComm_NVMe_loader/config/read_config.ini`:

| Key                                | Default                | Meaning                                                               |
|------------------------------------|------------------------|-----------------------------------------------------------------------|
| <code>read_duration_seconds</code> | <code>1.0</code>       | Seconds of the most recent capture to copy back when recording stops. |
| <code>sample_rate_hz</code>        | <code>800000000</code> | Receiver sample rate (Hz), used to convert duration into data size.   |
| <code>start_offset_seconds</code>  | <code>0.0</code>       | Offset (in seconds) from the start of the capture at which to begin.  |

The amount copied back is `seconds x sample_rate x 4 bytes` (each sample is one little-endian int16 I value followed by one little-endian int16 Q value), rounded to the nearest whole record and capped to the capture length. Example: 1 second at 800 MHz copies approximately 3,051.76 MiB.

### Setting Read-Back Parameters via gRPC

The parameters are configured from the **Client Computer** running the gRPC client:

```
# Set the read-back duration to 1.5 seconds and start offset to 0.5 seconds
python3 xrcomm_grpc_client.py --host <server-ip>:50051 set-read-capture 1.5 --start-offset 0.5

# Set the receiver sample rate to 800 MHz
python3 xrcomm_grpc_client.py --host <server-ip>:50051 set-read-rate 800000000
```

### Recording Workflow

```
python3 xrcomm_grpc_client.py --host <server-ip>:50051 set-logging on # Start
python3 xrcomm_grpc_client.py --host <server-ip>:50051 set-logging off # Stop -> auto read-back
```

### Output Files

Read-back files are written under the Platform Driver's working directory in `output_logs/readback_<timestamp>/:`

- `readback_<timestamp>_chunk_N.bin` – raw interleaved int16 little-endian IQ pairs (I, Q, I, Q, ...), split into 2 GB chunks.
- `readback_<timestamp>.idx` – a text manifest describing each block in the `.bin` files.

### Offline NVMe Retrieval

To extract older or specific data blocks manually after the platform has stopped:

```
cd XRComm_NVMe_loader/scripts
sudo ./read_nvme.sh
```

The underlying binary (`xrcomm_nvme_reader`) also accepts options for partial extraction:

| Option                                                     | Effect                                     |
|------------------------------------------------------------|--------------------------------------------|
| <code>--first-mb=&lt;N&gt; / --first-gb=&lt;N&gt;</code>   | Extract the first N megabytes / gigabytes. |
| <code>--middle-mb=&lt;N&gt; / --middle-gb=&lt;N&gt;</code> | Extract from the middle of the capture.    |



---

| Option                                                 | Effect                                                          |
|--------------------------------------------------------|-----------------------------------------------------------------|
| <code>--last-mb=&lt;N&gt; / --last-gb=&lt;N&gt;</code> | Extract the last N megabytes / gigabytes.                       |
| <code>--flash</code>                                   | Free the capture on the drive after successful full extraction. |
| <code>--force-flash</code>                             | Force delete even if extraction is partial or corrupted.        |

---

Extracted data uses the same format as the automatic read-back.

## Troubleshooting

| Symptom                                 | What to check                                                                                                                                      |
|-----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| No free hugepages on startup            | Clear stale files and reset the reservation (see Appendix A).                                                                                      |
| Platform fails to start / no interface  | Confirm NIC and NVMe binding (Appendix A). Check <code>systemctl status xrcomm-server</code> .                                                     |
| Application receives no data            | Confirm the platform is running and the matching mode is enabled. Check <code>list-ips</code> .                                                    |
| Full-packet example will not attach     | Start the platform first. Run only one full-packet application at a time.                                                                          |
| Permission denied                       | Ensure platform services are running. Try running the application with <code>sudo</code> .                                                         |
| Control client cannot reach the service | Check <code>systemctl status xrcomm-grpc-ctrl</code> . Verify port 50051 is not blocked by a firewall.                                             |
| Read-back produced no files             | Only triggers on <code>set-logging off</code> after a real capture. Confirm <code>set-read-rate</code> and <code>set-read-capture</code> were set. |
| Components disagree after update        | Re-run <code>install.sh</code> and restart all services so they share the same interface version.                                                  |

## Appendix A: Environment Prerequisites

These configurations are **prerequisites** for the FlexServer. They are typically set up once during initial server provisioning. If the platform fails to start or behaves unexpectedly, verify these settings first.

### Dependency Version Check Table

| Dependency                 | Required Version                | Installation Command / Source                               |
|----------------------------|---------------------------------|-------------------------------------------------------------|
| <b>Ubuntu Linux</b>        | 22.04 LTS or 24.04 LTS (64-bit) | Operating System standard install                           |
| <b>DPDK</b>                | v23.11.1                        | <code>sudo apt install dpdk libdpdk-dev</code>              |
| <b>SPDK</b>                | v24.01                          | Build from source                                           |
| <b>gRPC &amp; Protobuf</b> | v1.62.0 (C++ runtime)           | <code>sudo apt install libgrpc++-dev libprotobuf-dev</code> |
| <b>Python 3</b>            | v3.10+                          | Operating System standard install                           |
| <b>grpcio &amp; tools</b>  | v1.62.0 (Python runtime)        | <code>pip3 install grpcio grpcio-tools</code>               |
| <b>CMake</b>               | v3.16+                          | <code>sudo apt install cmake</code>                         |
| <b>GCC</b>                 | v11+                            | <code>sudo apt install build-essential</code>               |

## Hugepages

File-backed hugepages are required for zero-copy memory pools:

```
sudo rm -rf /dev/hugepages/* /mnt/huge/*           # Clear stale allocations
sudo sysctl -w vm.nr_hugepages=0
sudo sysctl -w vm.nr_hugepages=112                # Adjust to your memory layout (112 x 2MB)
grep -i huge /proc/meminfo                         # Verify
```

## Device Binding (DPDK / SPDK)

Before launching the server, the receive NIC interfaces and NVMe storage drive must be bound to the correct DPDK/SPDK user-space drivers:

- **Network Interface (DPDK Binding):** Bind the wideband receive NIC ports to the DPDK-compatible driver (vfio-pci) using the helper script:  

```
sudo modprobe vfio-pci
sudo dpdk-devbind.py --bind=vfio-pci 0000:6a:00.0 0000:6a:00.1
```
- **NVMe Drive (SPDK Binding):** Bind the recording NVMe PCIe drive to SPDK using the SPDK setup.sh script.

## Privileges

- The Platform Driver and the full-packet example require `sudo` (hugepages and direct device access).
- The IQ example and the control client do not require elevated privileges.

---

**Support:** For assistance, contact XRComm Inc. using the support address provided with your delivery package. Include the output of `systemctl status xrcomm-server`, `journalctl -u xrcomm-server.service --no-pager -n 50`, `get-stats`, and `get-mode-stats`.